

Phase 1 Draft: Text-to-Video Diffusion with a Lightweight 3D U-Net Foundation & Analysis

Rahma Hamdy / Ahmed Mamdouh / Rowan Hany / Haneen Ehab
AIE418

May 6, 2026

Abstract

We study denoising diffusion probabilistic models (DDPMs) for short text-conditioned video clips. This phase 1 draft is written to support the foundation review: a concrete creative theme, an explicit statement of which components are *architecturally* inherited from modern diffusion systems versus which weights are pretrained, a concise DDPM treatment (forward process, simplified loss, CFG, schedules, prediction targets), a implementation-grounded 3D U-Net description, and two weakness studies with planned metrics (temporal flicker; text–video alignment).

1 Introduction

1.1 Selected theme and motivation

Our creative theme is **Dreamlike Cities of the Future**: short cinematic clips in futuristic-urban settings, emphasizing coherent motion, lighting, and semantic alignment to the prompt (see `src/data/prompts.py` in the codebase). Example prompts include neon street scenes, aerial traffic above skyscrapers, and cyberpunk markets—chosen so that *temporal structure* (camera motion, drifting fog, traffic flow) is part of the intended content, which stress-tests temporal modeling.

More broadly, we study *text-to-video* generation: given a natural-language prompt, the system should produce a short clip that matches the semantics of the text while avoiding distracting frame-to-frame inconsistencies (flicker, object “jitter,” sudden appearance changes). We are motivated by the gap between strong image diffusion models and video: independent frame synthesis often looks plausible per frame yet fails as *motion* and *continuity*. Diffusion offers a principled generative framework; extending it to spatiotemporal tensors raises efficiency, conditioning, and temporal-stability questions that we treat as first-class design constraints.

1.2 Scope of this phase 1 draft

This document satisfies the week 13 foundation checklist and matches the topics a TA typically probes in an early session: motivation (§1), what is meant by a “pretrained model” in our setup (§3), mathematical grounding (§2), architecture details (§3), and weakness analysis with metrics (§4).

2 Background and Mathematics

2.1 Forward noising process (DDPM)

Let a clean video clip be a tensor $x_0 \in \mathbb{R}^{C \times T \times H \times W}$. DDPM defines a Markov forward process that gradually adds Gaussian noise over discrete steps $t = 1, \dots, T$:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad (1)$$

with variance schedule $\{\beta_t\}$. Using the reparameterization trick, one may sample x_t directly from x_0 :

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (2)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. As t grows, $\bar{\alpha}_t \rightarrow 0$ and x_t approaches pure noise.

2.2 Reverse denoising and score intuition

The generative model learns a reverse Markov chain $p_\theta(x_{t-1} | x_t, c)$ (with conditioning c , e.g. a text embedding) to denoise samples. In practice, p_θ is implemented by a neural network that predicts noise ϵ , clean x_0 , or a reparameterized target (see below). The *score* $\nabla_x \log p(x)$ points toward higher-density regions under the data distribution; denoising networks are closely related to learning such scores, and predicting noise under Gaussian perturbations is a standard scalable parameterization.

2.3 Training objective (ELBO and simplified loss)

Maximizing a variational lower bound (ELBO) on $\log p(x_0 | c)$ leads to a weighted sum of denoising terms. Following Ho et al. (DDPM), one uses a simplified objective: sample t , noise ϵ , form x_t , and minimize expected ℓ_2 error between the network prediction and a chosen target (e.g. ϵ). We do not reproduce the full ELBO algebra here; the key idea is that a weighted denoising autoencoder loss approximates gradual removal of noise consistent with the forward process.

Operational training step (what the code implements). Each minibatch draws a clean clip x_0 , samples a step t and Gaussian noise ϵ , constructs $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$, runs the network to obtain a prediction of the chosen target (here ϵ), and backpropagates MSE plus any auxiliary terms (e.g. temporal smoothness on the target). At sampling time, the network’s noise estimate is inserted into the DDPM or DDIM update to step from x_t toward x_{t-1} .

2.4 Prediction targets and loss functions

Let $f_\theta(x_t, t, c)$ be the network output. Common choices:

- **Noise (ϵ) prediction.** Target ϵ . Loss: $\mathcal{L}_\epsilon = \mathbb{E}[\|f_\theta(x_t, t, c) - \epsilon\|_2^2]$.
- **x_0 prediction.** Target x_0 . Equivalent reparameterizations map x_0 -prediction to an implied noise estimate.
- **v -prediction.** Target $v = \sqrt{\bar{\alpha}_t} \epsilon - \sqrt{1 - \bar{\alpha}_t} x_0$, which can stabilize training across noise levels.

Our default configuration uses **ϵ -prediction** (`diffusion.prediction_target: eps`).

2.5 Classifier-free guidance (CFG)

During training, conditioning is randomly dropped with probability p_{drop} so the model also sees unconditional inputs. At sampling time, one forms conditional and unconditional predictions and extrapolates:

$$\tilde{\epsilon} = \epsilon_{\text{unc}} + w(\epsilon_{\text{cond}} - \epsilon_{\text{unc}}), \quad (3)$$

with guidance scale $w \geq 1$. Larger w typically improves text alignment but may reduce diversity and cause artifacts.

2.6 Noise schedules

We implement **linear** and **cosine** β -schedules (`diffusion.beta_schedule`). Linear schedules are simple and common; cosine schedules slow the destruction of signal at early timesteps and often improve sample quality in image models. We default to **linear** in our base config but note cosine as an ablation axis for stability and fidelity.

2.7 Video-specific considerations

Naive *frame-wise* diffusion ignores temporal coupling: each frame may be sharp yet jointly inconsistent. Our model processes $x \in \mathbb{R}^{C \times T \times H \times W}$ with **3D convolutions** and **temporal self-attention** so that denoising at step t depends on neighboring frames. We operate in **pixel space** (no latent VAE) for transparency and implementation simplicity, at higher compute cost than latent video diffusion at equal resolution.

3 Model Architecture

3.1 “Pretrained model”: architecture versus checkpoint

Course materials often ask for a *selected pretrained model*; in our project we separate two notions:

- **Selected architecture (diffusion denoiser).** We use a **3D U-Net** denoiser (`VideoUNet3D`), the same broad family as DDPM/Imagen-style image U-Nets extended to video tensors: residual blocks, downsampling/upsampling, skip connections, and attention for mixing information across positions. The **denoiser weights are trained from scratch** on our dataset; we do not load a public text-to-video checkpoint.
- **Optional pretrained module (text side).** When `model.text_encoder` is set to `clip`, prompt conditioning uses a **pretrained CLIP text tower** (OpenCLIP weights) mapped into the U-Net via projection and addition to the timestep embedding. The default **hash** encoder is **not** pretrained: it is a lightweight hash embedding for fast iteration, with weaker semantics (a deliberate baseline discussed in §4).

pretrained CLIP text features are optional; the spatiotemporal denoiser follows standard diffusion U-Net design but is learned end-to-end in our setting.

3.2 Choice of denoiser

We adopt the `VideoUNet3D` backbone described above: encoder–decoder structure with skip connections, `ResBlock3D` stages, a middle stack with temporal self-attention, and a convolutional output head returning a tensor shaped like the noisy video input.

3.3 Timestep and text conditioning

A sinusoidal embedding of the diffusion index t is passed through an MLP to produce a timestep embedding. Text prompts are embedded with either a lightweight **hash bag-of-n-grams** encoder (default) or an optional **CLIP** text tower for stronger alignment. The text vector is projected and **added** to the timestep embedding (early fusion), then injected into each residual block via projection to channel space—rather than full cross-attention over spatial-temporal tokens. For classifier-free guidance, training randomly replaces the conditional embedding with an unconditional token with probability `train.cond_drop_prob` (e.g. 0.1), so the same network learns both branches used in Eq. (3) at inference.

3.4 Backbone details (implementation mapping)

- **Residual blocks:** ResBlock3D uses GroupNorm, SiLU, $3 \times 3 \times 3$ convolutions, and timestep scale-shift.
- **Pooling / expansion:** strided 3D convolution downsamples spatially; transposed convolution upsamples; skip connections concatenate encoder features.
- **Attention:** SelfAttention3D applies multi-head attention *over time* independently at each spatial location (memory-aware temporal attention).
- **Output head:** a final convolution predicts the network target (e.g. noise) with the same shape as x .

3.5 Sampling

Training uses the forward q -process; generation uses either **DDPM** stochastic sampling or **DDIM** fewer-step sampling. Our base configuration sets `sample.sampler: ddim` with `num_steps: 50` and `guidance_scale: 5.0`.

4 Weakness Analysis

We highlight two weaknesses that are concrete in text-to-video diffusion and measurable with standard metrics. For each, we state a hypothesis, a mitigation implemented in our repository, and how we quantify impact. **Replace placeholder values in Table 1 with results from your runs** (e.g. `python -m src.evaluate` on paired prediction/reference folders).

4.1 Weakness 1: Temporal flicker and weak temporal consistency

Observation. Generated clips exhibit high-frequency temporal variation: textures and edges “boil” between frames even when the prompt is static.

Hypothesis. The plain ϵ -objective does not explicitly penalize *changes* in the predicted target across time, so the optimizer may settle on per-frame denoising solutions that are not temporally smooth.

Mitigation. We add an auxiliary **temporal smoothness** penalty on the training target: a weighted ℓ_2 penalty on frame-to-frame differences of the prediction target (`train.temporal_smooth_weight`, default 0.02).

Metrics. We report a **temporal LPIPS** (average LPIPS between adjacent predicted frames; lower is smoother) and a **FVD proxy** comparing feature statistics of predicted vs. reference frames (lower is better; note this is a proxy, not full I3D-FVD).

4.2 Weakness 2: Weak text–video semantic alignment

Observation. The model ignores parts of the prompt or binds attributes incorrectly (wrong object/color/action), especially under the default hash text encoder.

Hypothesis. Hash embeddings provide limited semantic structure compared to contrastive language–image models; the conditioning signal is too weak for fine-grained alignment.

Mitigation. Swap `model.text_encoder` to **CLIP** and set `text_emb_dim: 512`; keep CFG enabled. Optionally increase guidance within reasonable limits to trade diversity for alignment.

Metrics. We report **CLIP similarity** between the prompt and generated frames (higher is better, requires `open-clip-torch`) and optionally **SSIM** / **PSNR** against reference clips when available.

Table 1: Example weakness study “Baseline” matches `configs/base.yaml` unless noted.

Setting	Temp. LPIPS ↓	FVD proxy ↓	CLIP-SIM ↑	Notes
Baseline (hash, no smooth)	–	–	–	
+ temporal smooth	–	–	–	weight 0.02
+ CLIP text (smooth on/off)	–	–	–	

4.3 Brief discussion

The two weaknesses interact: stronger alignment (high CFG, CLIP) can sometimes exacerbate temporal artifacts if the denoiser struggles to reconcile text constraints with smooth dynamics; joint tuning of guidance, steps, and smoothness weight is expected.